Workshop I Practical — IOTA Move Foundations

You've seen the slides. Now let's make Move objects come alive.

In this workshop, you'll ship a tiny "ScholarFlow" package that mints Grant objects for students. Along the way you'll:

- Publish a Move package to IOTA
- Receive and inspect an admin capability object
- Design an owned object and an event
- Implement a mint flow that turns a function call into a new on-chain object owned by a student

> **THE ONE THING YOU'LL IMPLEMENT**
>
> You will implement `scholarflow::grant::mint` yourself. The rest of the package scaffolding is provided here.

What we're building (mental model first)

- An `AdminCap` struct with `key` creates a real L1 object when instantiated. We'll hand this to the package publisher during `init`, so only they can mint in this workshop.
- A `Grant` is an owned object. When you instantiate it, it's created with a fresh `UID` and becomes transferable. Our goal is to transfer it to a student.
- A `GrantMinted` event captures what happened so indexers and UIs can react to mints.

> **WHAT IS THE KEY ABILITY?**
>
> - `key` means the struct can live on-chain as an object (it carries a `UID`). Instantiating it is like minting a serialized item with an owner.
> - Think of a boxed laptop with a serial number and a shipping label (owner address).

> **WHAT IS A CAPABILITY?**

> - A capability (like `AdminCap`) is a permission token; callers must present it (e.g., `&AdminCap`) to do privileged actions.
> - Like an office badge that opens a specific door; no badge, no entry.
> - Learn more: Capabilities pattern

If these concepts are new, skim: UID and ID and Using Events.

Prerequisites

- IOTA CLI installed and connected: Install, Connect
- A funded account: Get Test Tokens
- Optional: a second address to act as the student

Create a working directory:

```
mkdir -p ~/scholarflow && cd ~/scholarflow
```

Scaffold the package

```
iota move new scholarflow_core
cd scholarflow_core
```

And a minimal `Move.toml`:

```
[package]
name = "scholarflow_core"
version = "0.0.1"
edition = "2024"

[addresses]
scholarflow = "0x0"
iota = "0x2"
std = "0x1"
```

Useful references while you build:

- CLI usage: IOTA Move CLI
- Concepts: Module Initializers

Model the world in Move (small steps)

Create `sources/grant.move`. Start with the module, imports, and the three data types. Don't worry about functions yet.

```
module scholarflow::grant {
    use iota::object::{Self, UID, ID};
    use iota::tx_context::{Self, TxContext};
    use iota::transfer;
    use iota::event;

    /// Capability granting authority to mint grants.
    public struct AdminCap has key { id: UID }

    /// An owned grant object assigned to a student.
    public struct Grant has key {
        id: UID,
        student: address,
        amount: u64,
    }

    /// Emitted when a grant is minted.
    public struct GrantMinted has copy, drop, store {
        student: address,
        amount: u64,
        grant_id: ID,
    }
}
```

Notice how both `AdminCap` and `Grant` have the `key` ability. Instantiating either will create an object with a `UID` you can later transfer.

WHAT ARE UID AND ID?

- `UID` is the unique token stored inside the object that gives it identity; it isn't copyable and lives only within the object.
- `ID` is the canonical identifier value derived from a reference to the object (`object::id(&obj)`), great for events and lookups.
- Think: `UID` is the sealed certificate inside the box; `ID` is the serial number on the receipt.
- Learn more: UID and ID

Give someone the keys (module initializer)

Now, let's make sure the package publisher receives the admin capability when the package is published.

Append this function inside the same module:

```
    /// Runs once at package publish. Transfers AdminCap to the
publisher.
    fun init(ctx: &mut TxContext) {
        let cap = AdminCap { id: object::new(ctx) };
        let publisher = tx_context::sender(ctx);
        transfer::transfer(cap, publisher);
    }
```

WHAT ARE INIT AND TXCONTEXT?

- `fun init` runs once at publish time to bootstrap state (e.g., mint an `AdminCap`). It cannot be called again.
- `TxContext` is the per-transaction context: it gives you the sender and lets you create fresh object IDs (`object::new(ctx)`).
- Like a deployment migration that seeds initial rows and grants the first admin.
- Learn more: Module Initializers

Quick checkpoint:

```
iota move build
```

If you hit snags, see Build, Test, Debug and the Move CLI.

The mint you'll write

Here's the story your `mint` function should tell when it runs:

- Create a fresh `Grant` object with the intended `student` and an `amount`.
- Emit a `GrantMinted` event that includes the new object's `ID`.
- Transfer the newly created `Grant` to the `student` address.

We'll leave the full body for you to implement. Start with the signature and a placeholder, then iterate.

Append this inside the same module:

```
    public entry fun mint(
        student: address,
        amount: u64,
        _cap: &AdminCap,
```

```
        ctx: &mut TxContext
    ) {
        // TODO: Create Grant, emit event, transfer to `student`.
        // Hints: object::new(ctx), object::id(&grant),
transfer::transfer(grant, student)
        abort 0;
    }
```

> **ENTRY VS PUBLIC, AND TRANSFERS**
>
> - Both `public` and `entry` functions can be called from programmable transaction blocks (CLI/SDK/PTB). The difference is in constraints and visibility: `entry` adds stricter rules; `public` can be called from other modules.
> - Why `public entry` here: we want to call `grant::mint` from the CLI as a top-level transaction while keeping entry-function constraints (inputs must be direct tx arguments; returns must have `drop`). Marking it `public` also leaves room for other modules to reuse `mint` if needed later.
> - If you don't need inter-module reuse, an `entry` function (without `public`) is sufficient for CLI calls.
> - When minting/transferring objects you pass a `&mut TxContext` so the runtime can create IDs and record effects.
> - `transfer::transfer(obj, recipient)` moves ownership of the on-chain object to an address. One owner at a time, enforced by Move.
> - Learn more: Entry Functions • Object Ownership

Helpful references while implementing:

- Object IDs and `UID`: UID and ID
- Events: Using Events
- Entry functions: Entry Functions

Publish the package (and receive your cap)

```
iota client publish --gas-budget 100000000
```

Expected output (truncated example):

```
╭──────────────
```

> - Your gas coin is mutated to pay for the transaction; the package is published with its on-chain `PackageID`.

Export the `ADMIN_CAP_ID` for later commands (copy from Created Objects above):

```
export ADMIN_CAP_ID=0x<admin_cap_object_id>
```

Capture what publish returns and set environment vars for convenience:

```
export PKG_ID=<paste Package ID>
export ADMIN_CAP_ID=<paste AdminCap object ID>
export PUBLISHER_ADDR=<your publisher 0x...>

iota client objects --owner "$PUBLISHER_ADDR"
iota client object --id "$ADMIN_CAP_ID"
```

More context: Publish a Package

Try your mint (after you implement it)

```
export STUDENT_ADDR=<0x...student address...>
export AMOUNT=1000

iota client call \
  --package $PKG_ID \
  --module grant \
  --function mint \
  --args $STUDENT_ADDR $AMOUNT $ADMIN_CAP_ID \
  --gas-budget 100000000
```

Expected output (events excerpt):

```
╭─────────────────────────────────────────────────────────────────────────────
│ Transaction Block Events
├─────────────────────────────────────────────────────────────────────────────
│  ╭──
│  │ EventID: <event_id>
│  │ PackageID: 0x<package_id>
│  │ Transaction Module: grant
│  │ Sender: 0x<sender_address>
│  │ EventType: 0x<package_id>::grant::GrantMinted
│  │ ParsedJSON:
│  │   ╭──────────╮──────────────────────────────────────────────────────────
```

```
│   │   │ amount    │ <amount>
│   │   ├───────────┼──────────────────────────────────────────────────────────
│   │   │ grant_id  │ 0x<grant_object_id>
│   │   ├───────────┼──────────────────────────────────────────────────────────
│   │   │ student   │ 0x<student_address>
│   │   └───────────┴──────────────────────────────────────────────────────────
│   └──
```

Verify what happened

```
iota client objects --owner "$STUDENT_ADDR"
iota client object --id <GRANT_OBJECT_ID>
```

This confirms that a `Grant` object now exists and is owned by the student. For events, use your indexer of choice and the Using Events patterns. Ownership concepts recap: Object Ownership.

Handy script (optional)

```
#!/usr/bin/env bash
set -euo pipefail

: "${PKG_ID:?set PKG_ID}"
: "${ADMIN_CAP_ID:?set ADMIN_CAP_ID}"
: "${STUDENT_ADDR:?set STUDENT_ADDR}"
AMOUNT="${AMOUNT:-1000}"

iota client call \
  --package "$PKG_ID" \
  --module grant \
  --function mint \
  --args "$STUDENT_ADDR" "$AMOUNT" "$ADMIN_CAP_ID" \
  --gas-budget 100000000
```

What we learned

- Summary of what we built.
- To further practise, we suggest trying X,Y (Extension tasks)
- Add hints or extension tasks with the objective of deepening understanding.

Common pitfalls

> ⚠ **CAUTION**
>
> - Network: ensure your CLI is connected to Testnet (`iota client envs`; switch with `iota client switch --env testnet`).
> - Funding: request test tokens via faucet before publishing/calling (`iota client faucet --address $PUBLISHER_ADDR`).
> - First publish may rewrite addresses — re-check `Move.toml` and rebuild if you changed named addresses.
> - Make sure you pass the correct `ADMIN_CAP_ID` to `mint`.
> - Increase `--gas-budget` if you see out-of-gas failures.
> - Double-check signer and argument order for `iota client call`.

Bring this to Workshop II

Keep these handy:

- `PKG_ID`
- `ADMIN_CAP_ID`
- At least one `GRANT_OBJECT_ID`
- Publisher and student addresses

Further reading

- Get Address
- Programmable Transaction Basics
- Move Overview: Abilities
- CLI Cheatsheet

Give Feedback

Tags: Tutorial | Move Smart Contracts

Edit this page